

# CHAPTER 7

## Transformation

Transformations of the response and predictors can improve the fit and correct violations of model assumptions such as nonconstant error variance. We may also consider adding additional predictors that are functions of the existing predictors like quadratic or cross-product terms. This means we have more choice in choosing the transformations on the predictors than on the response.

### 7.1 Transforming the Response

We start with some general considerations about transforming the response. Suppose that you are contemplating a logged response in a simple regression situation:

$$\log y = \beta_0 + \beta_1 x + \epsilon$$

In the original scale of the response, this model becomes:

$$y = \exp(\beta_0 + \beta_1 x) \cdot \exp(\epsilon)$$

In this model, the errors enter *multiplicatively* and not *additively* as they usually do. So the use of standard regression methods for the logged response model requires that we believe the errors enter multiplicatively in the original scale. Notice that if we believe the true model for  $y$  to be:

$$y = \exp(\beta_0 + \beta_1 x) + \epsilon$$

then we cannot linearize this model and nonlinear regression methods would need to be applied.

As a practical matter, we usually do not know how the errors enter the model, additively, multiplicatively or otherwise. The typical approach is to try different transforms and then check the residuals to see whether they satisfy the conditions required for linear regression. Unless you have good information that the error enters in some particular way, this is the simplest and most appropriate approach.

Although you may transform the response, you will probably need to express predictions in the original scale. This is simply a matter of back transforming. For example, in the logged model above, your prediction would be  $\exp(\hat{y}_0)$ . If your prediction confidence interval in the logged scale was  $[l, u]$ , then you would use  $[\exp l, \exp u]$ . This interval will not be symmetric, but this may be desirable. For example, the untransformed prediction intervals for the Galapagos data went below zero in Section 3.5. Transformation of the response avoids this problem.

Regression coefficients will need to be interpreted with respect to the transformed scale. There is no straightforward way of back transforming them to values that can be

interpreted in the original scale. You cannot directly compare regression coefficients for models where the response transformation is different. Difficulties of this type may dissuade one from transforming the response even if this requires the use of another type of model, such as a generalized linear model.

When you use a log transformation on the response, the regression coefficients have a particular interpretation:

$$\begin{aligned}\log \hat{y} &= \hat{\beta}_0 + \hat{\beta}_1 x_1 + \cdots + \hat{\beta}_p x_p \\ \hat{y} &= e^{\hat{\beta}_0} e^{\hat{\beta}_1 x_1} \cdots e^{\hat{\beta}_p x_p}\end{aligned}$$

An increase of one in  $x_1$  would multiply the predicted response (in the original scale) by  $e^{\hat{\beta}_1}$ . Thus when a log scale is used, the regression coefficients can be interpreted in a multiplicative rather than an additive manner.

The Box-Cox method is a popular way to determine a transformation on the response. It is designed for strictly positive responses and chooses the transformation to find the best fit to the data. The method transforms the response  $y \rightarrow g_\lambda(y)$  where the family of transformations indexed by  $\lambda$  is:

$$g_\lambda(y) = \begin{cases} \frac{y^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \log y & \lambda = 0 \end{cases}$$

For fixed  $y > 0$ ,  $g_\lambda(y)$  is continuous in  $\lambda$ . Choose  $\lambda$  using maximum likelihood. The profile log-likelihood assuming normality of the errors is:

$$L(\lambda) = -\frac{n}{2} \log(\text{RSS}_\lambda/n) + (\lambda - 1) \sum \log y_i$$

where  $\text{RSS}_\lambda$  is the residual sum of squares when  $g_\lambda(y)$  is the response. You can compute  $\hat{\lambda}$  numerically to maximize this. If the purpose of the regression model is just prediction, then use  $y^\lambda$  as the response (no need to use  $(y^\lambda - 1)/\lambda$ , as the rescaling is just for convenience in maximizing the likelihood). If explaining the model is important, you should round  $\lambda$  to the nearest interpretable value. For example, if  $\hat{\lambda} = 0.46$  it would be hard to explain what this new response means, but  $\sqrt{y}$  might be easier.

Transforming the response can make the model harder to interpret so we do not want to do it unless it is really necessary. One way to check this is to form a confidence interval for  $\lambda$ . A  $100(1-\alpha)\%$  confidence interval for  $\lambda$  is:

$$\lambda: \quad L(\lambda) > L(\hat{\lambda}) - \frac{1}{2} \chi_1^2(1-\alpha)$$

This interval can be derived by inverting the likelihood ratio test of the hypothesis that  $H_0: \lambda = \lambda_0$  which uses the statistic  $2(L(\hat{\lambda}) - L(\lambda_0))$  having approximate null distribution

$\chi^2_1$ . The confidence interval also tells you how much it is reasonable to round  $\lambda$  for the sake of interpretability.

We check whether the response in the savings data need transformations. We will need the boxcox function from the MASS package:

```
> library (MASS)
```

Try it out on the savings dataset and plot the results:

```
> data(savings)
> g <- lm (sr ~ pop15+pop75+dpi+ddpi,savings)
> boxcox(g,plotit=T)
> boxcox (g, plotit=T, lambda=seq (0.5, 1.5, by=0.1))
```

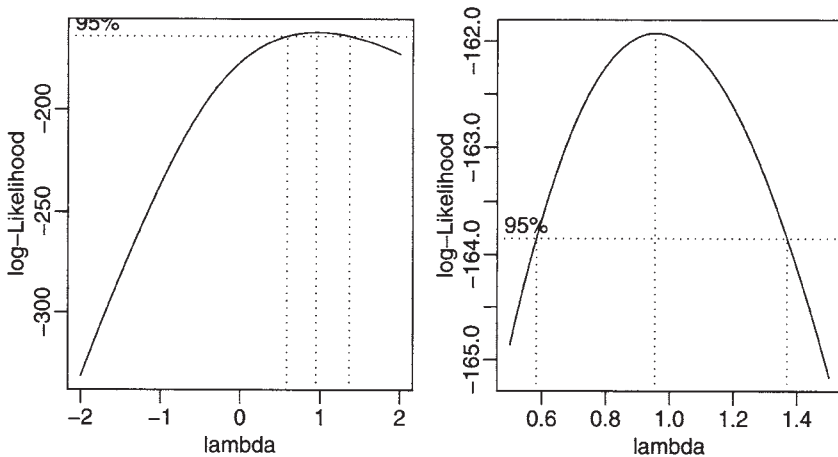


Figure 7.1 Log-likelihoodplots for the Box-Cox transformation of the savings data.

The first plot shown in Figure 7.1 is too broad. I narrowed the range of  $\lambda$  in the second plot so that we can read off the confidence interval more easily.

The confidence interval for  $\lambda$  runs from about 0.6 to about 1.4. We can see that there is no good reason to transform.

Now consider the Galápagos Islands dataset analyzed earlier:

```
> data (gala)
> g <- lm (Species ~ Area + Elevation + Nearest + Scrub +
  Adjacent, gala)
> boxcox (g, plotit=T)
> boxcox (g, lambda=seq (0.0, 1.0, by=0.05), plotit=T)
```

The plots are shown in Figure 7.2. We see that perhaps a cube root transformation might be best here. A square root is also a possibility, as this falls just within the confidence intervals. Certainly there is a strong need to transform.

Some general considerations concerning the Box–Cox method are:

1. The Box–Cox method gets upset by outliers—if you find  $\hat{\lambda}=5$ , then this is probably the reason—there can be little justification for actually making such an extreme transformation.
2. If some  $y_i < 0$ , we can add a constant to all the  $y$ . This can work provided the constant is small, but this is an inelegant solution.
3. If  $\max y_i / \min y_i$  is small, then the Box–Cox will not have much real effect because power transforms are well approximated by linear transformations over short intervals far from the origin.

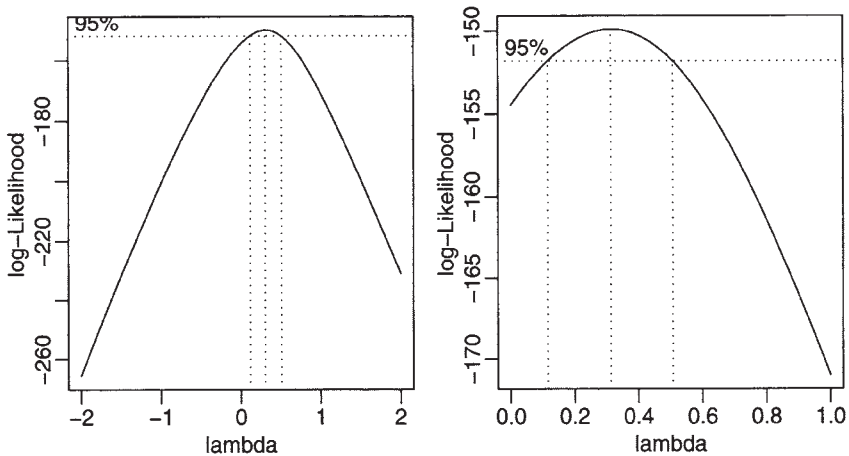


Figure 7.2 *Log-likelihood plots for the Box–Cox transformation of the Galápagos data.*

4. There is some doubt whether the estimation of  $\lambda$ , counts as an extra parameter to be considered in the degrees of freedom. This is a difficult question since  $\lambda$  is not a linear parameter and its estimation is not part of the least squares fit.

The Box–Cox method is not the only way of transforming the predictors. For responses that are proportions (or percentages), the logit transformation,  $\log(y/(1-y))$ , is often used, while for responses that are correlations, Fisher’s  $z$  transform,  $y=0.51\log((1+y)/(1-y))$ , is worth considering.

## 7.2 Transforming the Predictors

You can take a Box–Cox style approach for each of the predictors, choosing the transformation to minimize the RSS. However, this takes time. You can also use graphical methods such as partial residual plots to select transforming the predictors. These methods are designed to replace  $x$  in the model with  $f(x)$  for some chosen  $f$ . The

methods we consider below are more general in that they replace  $x$  with more than one term— $f(x)+g(x)+\dots$ . This allows more flexibility.

### 7.2.1 Broken Stick Regression

Sometimes we have reason to believe that different linear regression models apply in different regions of the data. For example, in the analysis of the savings data, we observed that there were two groups in the data and we might want to fit a different model to the two parts. Suppose we focus attention on just the `pop15` predictor for ease of presentation. We fit the two regression models depending on whether `pop15` is greater or less than 35%. The two fits are seen in Figure 7.3:

```
> g1 <- lm (sr ~ pop15, savings, subset=(pop15 < 35) )
> g2 <- lm (sr ~ pop15, savings, subset=(pop15 > 35) )
> plot (sr ~ pop15,savings,xlab="Pop'n under 15",
       ylab="Savings Rate")
> abline (v=35, lty=5)
> segments (20, g1$coef [1] +g1$coef[2]*20,35,
          g1$coef [1] +g1$coef [2] *35)
> segments (48, g2$coef [1] +g2$coef[2]*48,35,
          g2$coef [1] +g2$coef [2] * 35)
```

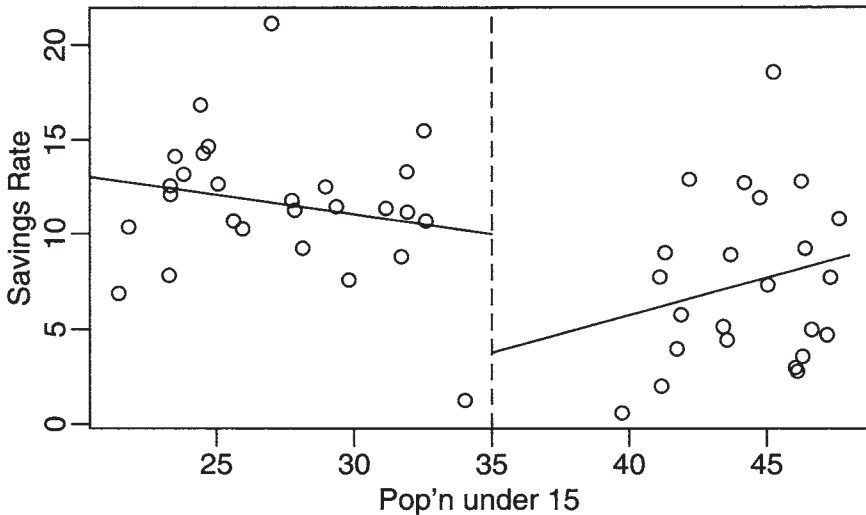


Figure 7.3 *Subset regression fit is shown with the solid line, while the broken stick regression is shown with the dotted line.*

A possible objection to this subsetted regression fit is that the two parts of the fit do not meet at the join. If we believe the fit should be continuous as the predictor varies, we should consider the broken stick regression fit. Define two *basis functions*:

$$B_l(x) = \begin{cases} c-x & \text{if } x < c \\ 0 & \text{otherwise} \end{cases}$$

and:

$$B_r(x) = \begin{cases} x-c & \text{if } x > c \\ 0 & \text{otherwise} \end{cases}$$

where  $c$  marks the division between the two groups.  $B_l$  and  $B_r$  form a first-order spline basis with a knotpoint at  $c$ . Sometimes  $B_l$  and  $B_r$  are called hockey-stick functions because of their shape. We can now fit a model of the form:

$$y = \beta_0 + \beta_1 B_l(x) + \beta_2 B_r(x) + \varepsilon$$

using standard regression methods. The two linear parts are guaranteed to meet at  $c$ . Notice that this model uses only three parameters in contrast to the four total parameters used in the subsetted regression illustrated before. A parameter has been saved by insisting on the continuity of the fit at  $c$ .

We define the two hockey-stick functions, compute and display the fit:

```
> lhs <- function (x) ifelse(x < 35, 35-x, 0)
> rhs <- function (x) ifelse(x < 35, 0, x-35)
> gb <- lm (sr ~ lhs (pop15) + rhs(pop15), savings)
> x <- seq(20, 48, by=1)
> py <- gb$coef[1]+gb$coef[2]*lhs(x)+gb$coef[3]*rhs(x)
> lines (x, py, lty=2)
```

The two (dotted) lines now meet at 35, as shown in Figure 7.3. The intercept of this model is the value of the response at the join.

We might question which fit is preferable in this particular instance. For the high pop15 countries, we see that the imposition of continuity causes a change in sign for the slope of the fit. We might argue that because the two groups of countries are so different and there are so few countries in the middle region, we might not want to impose continuity at all.

We can have more than one knotpoint simply by defining more basis functions with different knotpoints. Broken stick regression is sometimes called *segmented regression*. Allowing the knotpoints to be parameters is worth considering, but this will result in a nonlinear model.

### 7.2.2 Polynomials

Another way of generalizing the  $X\beta$  part of the model is to add polynomial terms. In the one-predictor case, we have:

$$y = \beta_0 + \beta_1 x + \dots + \beta_d x^d + \varepsilon$$

which allows for a more flexible relationship, although we usually do not believe it exactly represents any underlying reality.

There are two ways to choose  $d$ . We can keep adding terms until the added term is not statistically significant. Alternatively, we can start with a large  $d$  and eliminate nonstatistically significant terms starting with the highest order term.

Do *not* eliminate lower order terms from the model even if they are not statistically significant. An additive change in scale would change the t-statistic of all but the highest order term. We would not want the conclusions of our study to be so brittle to such changes in the scale which ought to be inconsequential.

Let's see if we can use polynomial regression on the `ddpi` variable in the savings data. First, fit a linear model:

```
> summary (lm (sr ~ ddpi, savings) )
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    7.883      1.011     7.80  4.5e-10
ddpi            0.476      0.215     2.22   0.031
```

The p-value of `ddpi` is significant so move on to a quadratic term:

```
> summary (lm (sr ~ ddpi+I (ddpi^2), savings))
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    5.1304      1.4347     3.58  0.00082
ddpi            1.7575      0.5377     3.27  0.00203
I (ddpi^2)     -0.0930      0.0361    -2.57  0.01326
```

Again the p-value of `ddpi`<sup>2</sup> is significant so move on to a cubic term:

```
> summary (lm (sr ~ ddpi+I (ddpi^2)+I (ddpi^3), savings))
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  5.145360      2.198606     2.34   0.024
ddpi          1.746017      1.380455     1.26   0.212
I (ddpi^2)   -0.090967      0.225598    -0.40   0.689
I (ddpi^3)   -0.000085      0.009374    -0.01   0.993
```

The p-value of `ddpi`<sup>3</sup> is not significant so stick with the quadratic. Notice how the other p-values are not significant in contrast with earlier results. Note that starting from a large model (including the fourth power) and working downwards gives the same result.

To illustrate the point about the significance of lower order terms, suppose we transform `ddpi` by subtracting 10 and refit the quadratic model:

```
> savings <- data.frame (savings, mddpi=savings$ddpi-10) >
summary (lm (sr ~ mddpi+I (mddpi^2), savings))
Coefficients:
```

|             | Estimate | Std. Error | t value | Pr(> t ) |
|-------------|----------|------------|---------|----------|
| (Intercept) | 13.4070  | 1.4240     | 9.41    | 2.2e-12  |
| mddpi       | -0.1022  | 0.3027     | -0.34   | 0.737    |
| I(mddpi^2)  | -0.0930  | 0.0361     | -2.57   | 0.013    |

We see that the quadratic term remains unchanged, but the linear term is now insignificant. Since there is often no necessary importance to zero on a scale of measurement, there is no good reason to remove the linear term in this model but not in the previous version. No advantage would be gained.

You have to refit the model each time a term is removed and for large  $d$  there can be a problem with numerical stability. Orthogonal polynomials get around this problem by defining:

$$\begin{aligned}z_1 &= a_1 + b_1 x \\z_2 &= a_2 + b_2 x + c_2 x^2 \\z_3 &= a_3 + b_3 x + c_3 x^2 + d_3 x^3\end{aligned}$$

etc. where the coefficients  $a, b, c, \dots$  are chosen so that  $z_i^T z_j = 0$  when  $i \neq j$ . The  $z$  are called orthogonal polynomials. The value of orthogonal polynomials has declined with advances in computing speeds although they are still worth knowing about because of their numerical stability and ease of use. The `poly()` function constructs orthogonal polynomials:

```
> g <- lm(sr ~ poly(ddpi, 4), savings)
> summary(g)
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   9.6710     0.5846   16.54 <2e-16
poly(ddpi, 4)1  9.5590     4.1338    2.31  0.025
poly(ddpi, 4)2 -10.4999     4.1338   -2.54  0.015
poly(ddpi, 4)3  -0.0374     4.1338   -0.01  0.993
poly(ddpi, 4)4   3.6120     4.1338    0.87  0.387
Residual standard error: 4.13 on 45 degrees of freedom
Multiple R-Squared:  0.218, Adjusted R-squared:  0.149
F-statistic: 3.14 on 4 and 45 DF, p-value: 0.0232
```

We can come to the same conclusion as above, that the quadratic model is best, with just this summary.

You can also define polynomials in more than one variable. These are sometimes called *response surface* models. A second degree model would be:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_{11} x_1^2 + \beta_{22} x_2^2 + \beta_{12} x_1 x_2$$

an example of which could be fit as:

```
> g <- lm (sr ~ poly (popl5, ddpi, degree=2), savings)
```

### 7.2.3 Regression Splines

Polynomials have the advantage of smoothness, but the disadvantage that each data point affects the fit globally. This is because the power functions used for the polynomials take nonzero values across the whole range of the predictor. In contrast, the broken stick regression method localizes the influence of each data point to its particular segment which is good, but we do not have the same smoothness as with the polynomials. There is a way we can combine the beneficial aspects of both these methods—smoothness and local influence—by using *B-spline* basis functions.

We may define a cubic B-spline basis on the interval  $[a, b]$  by the following requirements on the interior basis functions with knotpoints at  $t_1, \dots, t_k$ :

1. A given basis function is nonzero on an interval defined by four successive knots and zero elsewhere. This property ensures the local influence property.
2. The basis function is a cubic polynomial for each subinterval between successive knots.
3. The basis function is continuous and is also continuous in its first and second derivatives at each knotpoint. This property ensures the smoothness of the fit.
4. The basis function integrates to one over its support.

The basis functions at the ends of the interval are defined a little differently to ensure continuity in derivatives at the edge of the interval. A full definition of B-splines and more details about their properties may be found in de Boor (2002). The broken stick regression is an example of the use of linear splines.

Let's see how the competing methods do on a constructed example. Suppose we know the true model is:

$$y = \sin^3(2\pi x^3) + \epsilon, \quad \epsilon \sim N(0, (0.1)^2)$$

The advantage of using simulated data is that we can see how close our methods come to the truth. We generate the data and display them in the first plot of Figure 7.4.

```
> funky <- function (x) sin (2*pi*x^3) ^3
> x <- seq (0, 1, by=0.01)
> y <- funky (x) + 0.1*rnorm (101)
> matplot (x, cbind (y, funky (x)), type="pl", ylab="y",
  pch=18, lty=1)
```

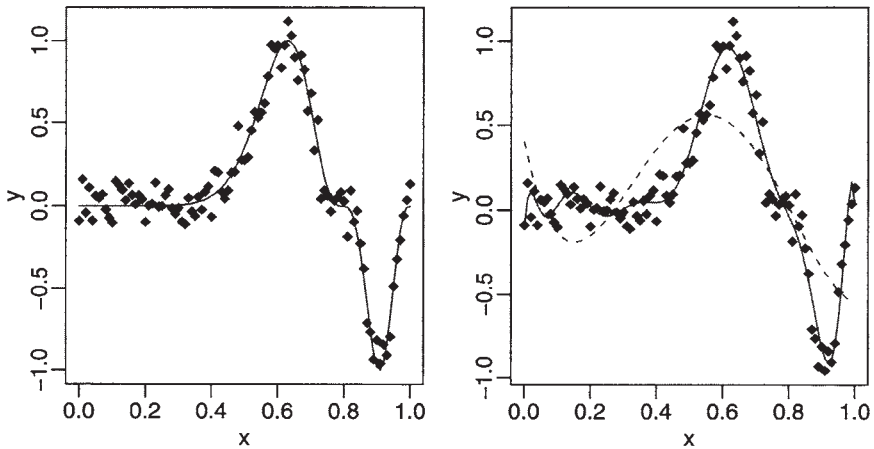


Figure 7.4 Data and true function shown on the left. Orthogonal polynomial of order 4 (dashed) and order 12 (solid) shown on the right.

We see how orthogonal polynomial bases of orders 4 and 12 do in fitting these data:

```
> g4 <- lm (y ~ poly(x,4))
> g12 <- lm (y ~ poly(x,12))
> matplot (x, cbind(y, g4$f.it, g12$f.it) , type="p11,
  ylab="y", pch=18,lty=c (1,2))
```

The two fits are shown in the second panel of Figure 7.4. We see that order 4 is a clear underfit; order 12 is much better although the fit is too wiggly in the first section and misses the point of inflection around  $x=0.8$ .

We now create the B-spline basis. You need to have three additional knots at the start and end to get the right basis. I have chosen to the knot locations to put more in the regions of greater curvature. I have used 12 basis functions for comparability to the orthogonal polynomial fit:

```
> library (splines)
> knots <- c (0, 0, 0, 0, 0.2, 0.4, 0.5, 0.6, 0.7, 0.8,
  0.85, 0.9, 1, 1, 1, 1)
> bx <- splineDesign (knots, x)
> gs <- lm (y ~ bx)
> matplot (x, bx, type="l")
> matplot (x, cbind (y, gs$fit), type="pl", ylab="y",
  pch=18, lty=1)
```

The basis functions themselves are seen in the first panel of Figure 7.5 while the fit itself appears in the second panel. We see that the fit comes very close to the truth.

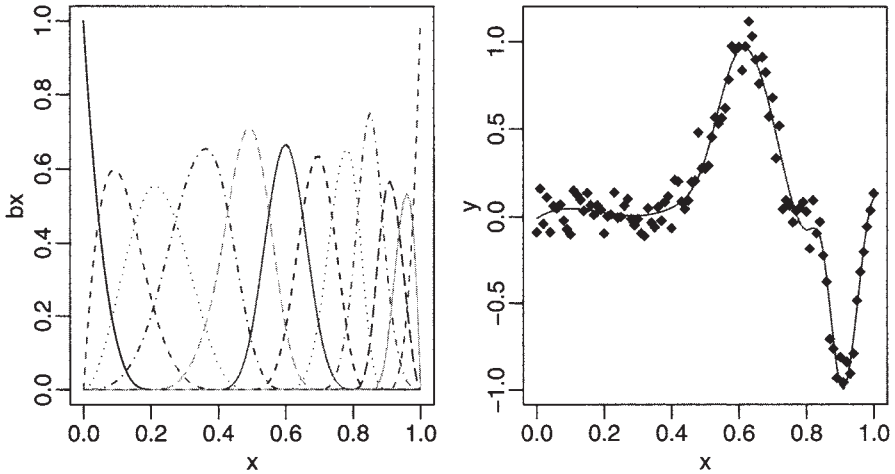


Figure 7.5 *Cubic spline basis function on the left, cubic spline fit to the data on the right.*

Regression splines are useful for fitting functions with some flexibility provided we have enough data. We can form basis functions for all the predictors in our model but we need to be careful not to use up too many degrees of freedom.

#### 7.2.4 Overview

The methods described above are somewhat awkward to apply exhaustively and even then they may miss important structure because of the problem of trying to find good transformations on several variables simultaneously. One recent approach is the additive model:

$$y = \beta_0 + f_1(x_1) + f_2(x_2) + \dots + f_p(x_p) + \varepsilon$$

where nonparametric regression techniques are used to estimate the  $f_i$ s. Alternatively, you could implement this using the regression spline bases for each predictor variable. Other techniques are ACE, AVAS, regression trees, MARS and neural networks. See, for example, Ripley (1996) and Hastie, Tibshirani, and Friedman (2001) among many others.

It is important to realize the strengths and weaknesses of regression analysis. For larger datasets with relatively little noise, more recently developed complex models will be able to fit the data better while keeping the number of parameters under control. For smaller datasets or where the noise level is high (as is typically found in the social sciences), more complex models are not justified and standard regression is most effective. One relative advantage of regression is that the models are easier to interpret in contrast to techniques like neural networks which are usually only good for predictive purposes.

#### Exercises

1. The aatemp data come from the U.S. Historical Climatology network. They are the annual mean temperatures (in degrees F) in Ann Arbor, Michigan going back about 150 years.

- (a) Is there a linear trend?
  - (b) Observations in successive years may be correlated. Fit a model that estimates this correlation. Does this change your opinion about the trend?
  - (c) Fit a polynomial model with degree 10 and use backward elimination to reduce the degree of the model. Plot your fitted model on top of the data. Use this model to predict the temperature in 2020.
  - (d) Suppose someone claims that the temperature was constant until 1930 and then began a linear trend. Fit a model corresponding to this claim. What does the fitted model say about this claim?
  - (e) Make a cubic spline fit with six basis functions evenly spaced on the range. Plot the fit in comparison to the previous fits. Does this model fit better than the straight-line model?
2. The cornnit data on the relationship between corn yield (bushels per acre) and nitrogen (pounds per acre) fertilizer application were studied in Wisconsin in 1994. Use transformations to find a good model for predicting yield from nitrogen. Use a goodness of fit test to check your model.
  3. Using the ozone data, fit a model with O3 as the response and temp, humidity and ibh as predictors. Use the Box–Cox method to determine the best transformation on the response.
  4. Using the pressure data, fit a model with pressure as the response and temperature as the predictor using transformations to obtain a good fit.
  5. Use transformations to find a good model for volume in terms of girth and height using the trees data.

